

Framework và công cụ (Frameworks and Tools)

PHỤ THUỘC Toàn bộ cây — chương này chỉ có nghĩa khi bạn đã biết mình đang tìm gì. Đọc nó trước khi đọc Phần A là lãng phí.

TÓM TẮT

Chương này là bản đồ công cụ, và nó được viết xoay quanh một câu hỏi duy nhất: *đọc mã nguồn nào để học được gì?* Cách tổ chức ấy là có chủ ý — một danh sách dự án hết hạn trong hai năm, nhưng nguyên tắc “muốn hiểu marginalization thì đọc VINS-Mono” thì bền hơn nhiều. Nội dung gồm: nguyên tắc chọn mã nguồn để đọc, và vì sao đọc mã dạy nhiều hơn đọc bài báo với phần lớn nội dung Phần C và Phần D; sáu hệ thống đáng đọc trọn vẹn, mỗi hệ kèm chỉ dẫn *đọc phần nào trước*; các thư viện nền và tiêu chí chọn giữa chúng; và bộ dữ liệu cùng công cụ đánh giá. Nếu chỉ nhớ một điều: *phần lớn kiến thức ở Phần C và Phần D không tồn tại trong bài báo* — kiến trúc ba luồng, chính sách quản lý bản đồ, các cổng và ngưỡng — và cách duy nhất học chúng là đọc mã của một hệ đã trưởng thành.

CẠM BẮY

Chương này chót tại tháng 9/2026 và là phần hết hạn nhanh nhất của cây cùng với C/16 và C/17. Dự án đổi, API đổi, hệ mới xuất hiện. Rà lại mỗi sáu tháng. Tôi cũng **không xếp hạng chất lượng** các dự án. Lý do: “tốt hơn” phụ thuộc vào việc bạn cần gì, và một bảng xếp hạng sẽ sai ngay khi ai đó có ràng buộc khác tôi. Thay vào đó, mỗi mục trả lời câu “*đọc nó để học được gì*” — một câu hỏi có câu trả lời ổn định hơn nhiều.

1.1 Vì sao đọc mã nguồn

Có một sự bất đối xứng đáng nói giữa hai nửa của cây này.

Với Phần A và phần lớn Phần B — hình học, MAP, bundle adjustment, các paradigm back-end — *bài báo là nguồn tốt*. Nội dung là toán, nó viết ra được, và một bài báo tốt trình bày nó rõ hơn mã nguồn.

Với Phần C và Phần D thì ngược lại, và lý do có cấu trúc. Kiến trúc ba luồng ở C/13 mục 4A, chính sách chọn và loại keyframe, các cổng ở B/10 và B/12, cách hoà nhập loop closure với tracking — những thứ ấy *không viết được thành định lý*, chúng không phải đóng góp mà một bài báo khoe, và chúng chiếm phần lớn mã nguồn. Một bài báo mô tả hệ thống như thể nó chạy tuần tự; mã nguồn thật đầy hàng đợi, khoá, và cờ trạng thái.

Hệ quả thực hành: *với Phần C và Phần D, đọc mã là cách học chính, không phải cách bổ sung*. Và đọc mã của một hệ *đã trưởng thành* — đã chạy trên nhiều bộ dữ liệu, đã được nhiều người dùng, đã sửa nhiều lỗi — dạy nhiều hơn đọc mã của một hệ mới hơn nhưng chưa va chạm.

Cách đọc mã có hiệu quả, ba nguyên tắc.

Một — đọc theo câu hỏi, không đọc tuần tự. Mở mã với một câu hỏi cụ thể: “nó quyết định tạo keyframe thế nào?” Tìm chỗ ấy, đọc kỹ, rồi đóng lại. Đọc từ đầu tới cuối một hệ vài chục nghìn dòng là cách chắc chắn bỏ cuộc.

Hai — tìm các ngưỡng và hỏi chúng từ đâu ra. Mỗi hằng số trong mã là một quyết định. Phần lớn được chỉnh bằng thực nghiệm, và biết cái nào có cơ sở lý thuyết và cái nào không là kiến thức có giá trị.

Ba — so mã với bài báo của chính nó. Chỗ chúng khác nhau là chỗ thú vị nhất: đó là chỗ lý thuyết gặp thực tế, và thường là chỗ có một bài học không được viết ra.

1.2 Sáu hệ thống đáng đọc

Bảng 1.1. Sáu hệ thống và cái đáng học từ mỗi hệ. Cột cuối là chỉ dẫn đọc — phần nào trước.

Hệ thống	Đọc để học	Đọc phần nào trước
ORB-SLAM3	Kiến trúc SLAM đầy đủ nhất: ba luồng, covisibility, Atlas, quản lý bản đồ	Bắt đầu ở vòng lặp tracking; rồi chính sách tạo và loại keyframe (C/13); rồi loop closing và cách nó hoà nhập với tracking
VINS-Fusion	Cửa sổ trượt và marginalization, ngắn và dễ đọc nhất trong sáu	Bắt đầu ở chỗ dựng bài toán tối ưu; rồi marginalization (A/04 mục 5); rồi khởi tạo VI
OpenVINS	Bộ lọc chuẩn mực; hiệu chuẩn trực tuyến; có mô phỏng có chân trị	Chỗ định nghĩa vector trạng thái; rồi cờ FEJ và chỗ Jacobian được tính; rồi mô-đun mô phỏng — đây là chỗ tính NEES được (C/18)
GTSAM	Đồ thị nhân tử đúng cách; iSAM2; cách viết một loại nhân tử mới	Ví dụ về custom factor; rồi cấu trúc Bayes tree; rồi Marginals (A/06 mục 2B)
Ceres	Manifold, auto-diff, bộ giải thừa với Schur	Tài liệu về Manifold (A/03 mục 6); rồi các lựa chọn linear solver và ordering (A/04 mục 4D)
Kalibr	Toàn bộ tư duy hiệu chuẩn = batch MAP	Tài liệu quy trình thu dữ liệu trước mã — mỗi yêu cầu trong đó là một hàng của Bảng kích thích ở A/05

Một gợi ý về thứ tự nếu bạn đọc cả sáu: *VINS-Fusion* trước vì nó ngắn nhất và cấu trúc rõ; *OpenVINS* tiếp theo để thấy thế giới bộ lọc và để có mô phỏng; *ORB-SLAM3* thứ ba vì nó đầy đủ nhất và cũng phức tạp nhất — đọc nó trước khi có nền là nản. *Kalibr* đọc song song khi học C/14. *GTSAM* và *Ceres* đọc như tài liệu tham khảo khi cần, không đọc tuần tự.

1.3 Các hệ thống khác, theo chủ đề

Xếp theo chủ đề bạn muốn học chứ không theo mức độ nổi tiếng.

Direct method và mô hình quang trắc: *DSO* là cài đặt tham chiếu; *DM-VIO* thêm quán tính và dynamic marginalization. Đọc *DSO* để hiểu B/08 mục 2B và A/02 mục 5B.

Kỹ thuật số học và chất lượng phần mềm: [Basalt](#). Dạng căn bậc hai (B/09 mục 5A), spline $SE(3)$ cho hiệu chuẩn, và chất lượng kỹ thuật rất cao — đáng đọc như một ví dụ về cách viết phần mềm SLAM tử tế.

Semi-direct và tốc độ: [SVO-Pro](#). Đọc để thấy mục 2C của B/08 — phân công theo thể mạnh.

SfM và localization quy mô lớn: [COLMAP](#) cho SfM chuẩn công nghiệp; [HLoc](#) cho pipeline localization phân cấp (B/12 mục 2).

Kiến trúc lai học được: [DROID-SLAM](#) và [DPVO](#). Đọc để thấy ranh giới ở C/17 mục 3B được đặt ở đâu trong mã.

Quản lý bản đồ nhiều phiên: [maplab](#). Hiếm framework coi C/13 là chủ đề chính chứ không phải phụ.

Ngữ nghĩa trong đồ thị: [Kimera](#). Ví dụ về việc thông tin ngữ nghĩa cũng chỉ là một loại nhân tử.

Biểu diễn neural: [SpliTAM](#), [MonoGS](#). Chốt 9/2026; xem cảnh báo ở C/16 mục 4.

Fiducial: [TagSLAM](#). Liên quan `.../marker/`.

1.4 Thư viện nền

Lie group: [Sophus](#) là lựa chọn phổ biến nhất trong hệ sinh thái C++ SLAM; [manif](#) có API hiện đại hơn và Jacobian tường minh cho mọi phép toán, và nó đi kèm [1] vốn là tài liệu tham chiếu tốt nhất về quy ước. Tiêu chí chọn: nếu bạn đang học và cần hiểu quy ước rõ ràng, dùng [manif](#); nếu bạn đang tích hợp vào hệ sinh thái có sẵn, dùng cái mà hệ đó dùng.

Tối ưu: [Ceres](#) cho auto-diff và sự trưởng thành; [GTSAM](#) cho đồ thị nhân tử và smoothing gia tăng; [g2o](#) cho mã ngắn dễ đọc (tốt để học, không nhất thiết để dùng); [SymForce](#) cho việc sinh mã Jacobian tối ưu; [Theseus](#) cho tối ưu khả vi (C/17 mục 3B).

Tiêu chí chọn giữa Ceres và GTSAM, vốn là lựa chọn hay gặp nhất: nếu bài toán của bạn là một bài toán tối ưu tĩnh — hiệu chuẩn, BA ngoại tuyến — thì Ceres đơn giản hơn. Nếu nó là một bài toán *gia tăng* với các biến được thêm liên tục — SLAM thời gian thực — thì GTSAM với iSAM2 có lợi thế cấu trúc.

Thị giác: [OpenCV](#) cho gần như mọi thứ ở B/07 và A/02. Một cảnh báo thực dụng: kiểm xem build của bạn có bật đúng cờ kiến trúc (SIMD) không — đây là một lỗi hiệu năng phổ biến và dễ sửa (D/20 mục 4B).

1.5 Bộ dữ liệu và công cụ đánh giá

Công cụ: [evo](#) dễ dùng; [rpg_trajectory_evaluation](#) [2] chặt chẽ hơn về căn chỉnh. Với cả hai, chọn đúng chế độ căn chỉnh theo C/18 mục 2B — mặc định không phải lúc nào cũng đúng cho cấu hình của bạn, và đây là lỗi phổ biến nhất trong đánh giá.

Bộ dữ liệu, xếp theo cái chúng dùng để đo:

Đo độ chính xác trong điều kiện tốt: EuRoC [3] (drone, trong nhà, có motion capture), TUM-VI [4] (cầm tay, có hiệu chuẩn quang trắc), TUM-RGBD [5] (RGB-D, định nghĩa gốc của ATE/RPE), KITTI [6] (xe, ngoài trời).

Đo hoạt động dài hạn và thay đổi môi trường: OpenLORIS [7] — một trong rất ít bộ chạm vào C/13 mục 4C.

Đo localization trong điều kiện thay đổi: Aachen Day-Night [8].

CẦN NHỚ

Không bộ dữ liệu công khai nào đủ, và đây là nhắc lại D/21 mục 4A vì nó là điều quan trọng nhất của mục này.

Ba lý do: chúng không chứa các chế độ hỏng ở D/19; chúng quá ngắn cho các vấn đề ở C/13; và chúng đo ATE trung bình trên các chuỗi hệ thống chạy được, trong khi sản phẩm cần tỉ lệ thất bại và hành vi ở đuôi.

Dùng chúng cho đúng việc: *so sánh thuật toán trong điều kiện kiểm soát, và phát hiện hồi quy*. Không dùng chúng để kết luận hệ thống sẵn sàng cho khách hàng.

Việc cần làm song song: xây bộ dữ liệu nội bộ theo ba phần ở D/21 mục 4A — chuỗi đại diện, chuỗi nghịch cảnh, và chuỗi từ hiện trường. Phần thứ ba có giá trị nhất và chỉ có được nhờ vòng lặp fleet analytics.

1.6 Kết nối

Chương này là tham khảo cho toàn cây, và cách dùng đúng là tra ngược: khi đọc một chương và muốn xem nó được cài đặt thế nào, quay lại Bảng 1.1 và mục 3.

Nói xuống dưới: E/24-lo-trình giai đoạn hai là chỗ chương này được dùng nhiều nhất — đọc ba hệ thống theo thứ tự, và với mỗi hệ vẽ lại đồ thị nhân tử cùng danh sách biến nào bị fix. Đó là bài tập biến việc đọc mã từ tiêu thụ thụ động thành một việc có sản phẩm kiểm được.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao với Phần C và Phần D thì đọc mã là cách học chính chứ không phải bổ sung?
2. Nêu ba nguyên tắc đọc mã nguồn có hiệu quả, và nói nguyên tắc nào tiết kiệm nhiều thời gian nhất.
3. Với mỗi hệ trong sáu hệ ở Bảng 1.1, nói nó dạy được gì mà năm hệ kia không dạy.
4. Tiêu chí chọn giữa Ceres và GTSAM là gì?
5. Vì sao không bộ dữ liệu công khai nào đủ, và chúng nên được dùng cho việc gì?
6. Khi dùng `evo` hoặc `rpg_trajectory_evaluation`, lỗi phổ biến nhất là gì?

Nguồn

- [1] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018, mức tin B; tiền ấn nhưng được dùng rộng rãi làm tài liệu tham chiếu.
- [2] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.
- [3] M. Burri, J. Nikolic, P. Gohl, et al., “The EuRoC Micro Aerial Vehicle Datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016, mức tin A.
- [4] D. Schubert, T. Goll, N. Demmel, et al., “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A, 2018.
- [5] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của định nghĩa ATE và RPE, 2012.
- [6] A. Geiger, P. Lenz, and R. Urtasun, “Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2012.
- [7] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [8] T. Sattler, W. Maddern, C. Toft, et al., “Benchmarking 6DOF Outdoor Visual Localization in Changing Conditions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; bộ Aachen Day-Night, 2018.